UNIVERSITY OF PASSAU

Faculty of Computer Science and Mathematics

Reverse Engineering



Prof. Dr.-Ing. Christian Hammer

WS 2026/27

Disclaimer

- Reverse engineering may be illegal depending on the purpose and situation
- The information in this class is for educational purposes only
- Intended only in the context of this class (understanding of legacy code for which you have or are given permission by copyright owner to reverse engineer)
- Any reverse engineering without permission of copyright owner may have legal consequences
- It is the responsibility of each student to ensure legal abidance outside of this class
- Please refer to Directive 2009/24/EC (Computer Programs Directive) and national implementation laws.

- Origin: Hardware Analysis
- Disassemble product to understand its inner workings
- Goal: Specification for novel parts of e.g. competitors
- To copy product or adapt/fix a system

- Implication: Potential copyright infringement (patent, intellectual property)

- But: Repairing an owned product by disassembling it and replacing a part usually legal.

Reverse Engineering

- Supports *understanding* of a system through identification of the *components* or *artifacts* of the system
- Discovering *relationships* between them
- Generating *abstractions* of that information
- Goal: no alteration of the system in any way

- Abstractions: similar to SE

- Purpose: maintenance, alteration, or abuse

- E. J. Chikofsky and J. H. Cross, “Reverse Engineering and Design Recovery - A Taxonomy”, IEEE Software, Jan 1990, 13-17.

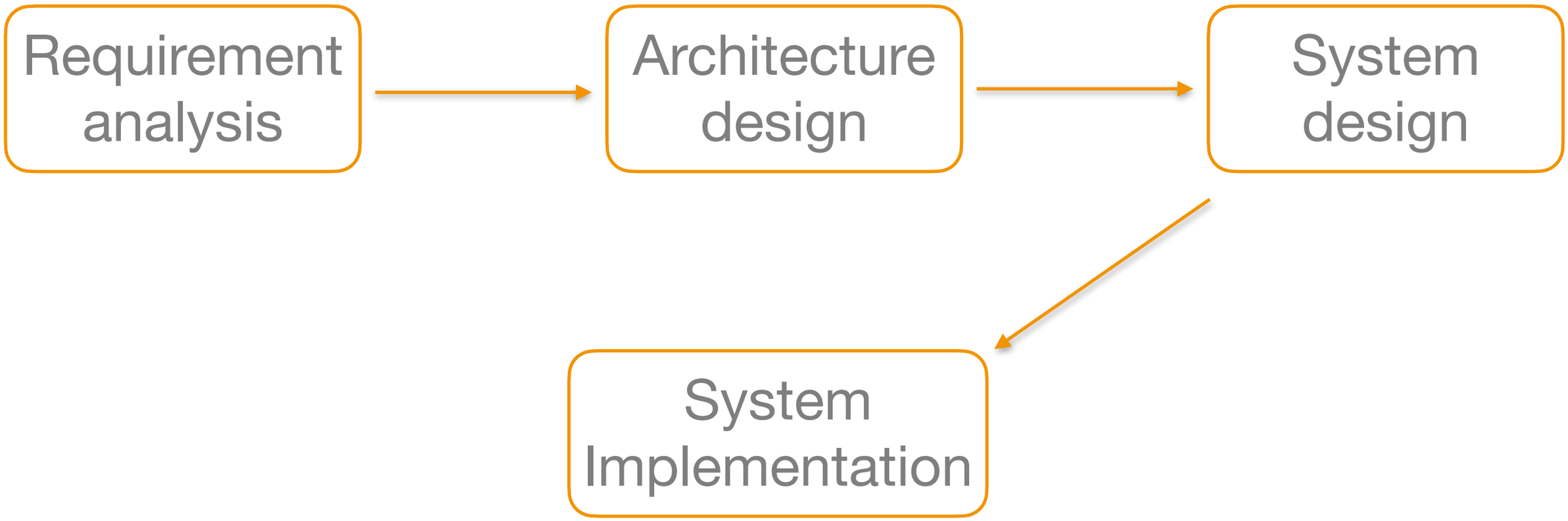
Goals of Reverse Engineering

- discover underlying system features: requirements, specifications, design and implementation

- system structure: components, their interrelationships (interfaces)
- functionality: operations performed (on what components)
- dynamic behavior: how input is transformed to output
- rationale: decision making between a number of alternatives at each design step
- construction: modules, documentation, test suites, etc.

- Understand semantics of program
  - Reengineering
  - Find/fix bugs, quality issues or defects (own software or third-party)
  - Security audits (e.g. untrusted compiler)
  - Management issues (programming standard)
  - Forensic analysis of malware (what, who, countermeasures)
  - Protocol/API analysis
  - Learning design decisions/ programming practices
  - Discovering underlying business model
  - Identification of intellectual property infringements

- Reversing forward engineering



- Problem: lost information

- Identify a system's artifacts and their relationships
- Techniques:
  - Dynamic Program Analysis and Profiling
  - Static Program Analysis
  - Informal extraction: other information sources
    - Potentially: Domain Knowledge (from involved developers etc.)
    - Documentation
    - Searching other sources (internet, etc.)

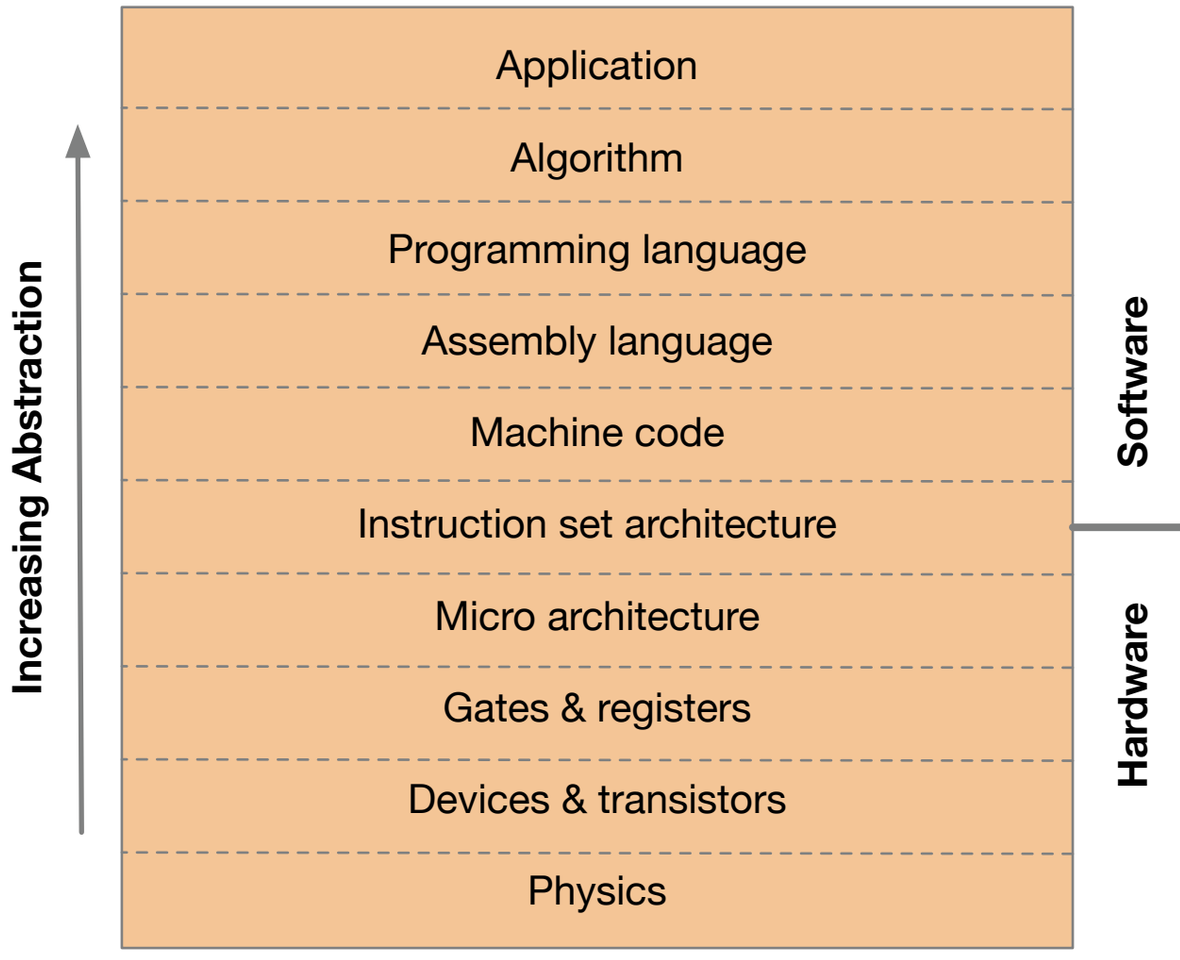
- Create model of the software (-> SE)
- Efficient storage of information
- Data structures allowing automated analysis
- Appropriate abstraction level

- Navigate through links in data structures (mostly manual)
  - Hierarchies, control flow, data flow, dependences
- Analysis (beyond basic model)
  - Patterns, domain knowledge
- Presentation
  - Visualization, etc.

Reverse Engineering Levels

- Worst case: only binary/object code
- Higher level: bytecode/intermediate representation
- Best case: Source code (might still be cryptic)

- Common goal: IR suitable for further analysis



- Even if source code is available: correspondence of source and binary?

- Understanding of computer organization, low level code
- Executable formats
- Interface to OS/HW
- Assembly

- Code level
  - Low-level, machine code or assembly
  - Recognize design, data structures, algorithms
    lists, trees, tables, vectors, matrices, dictionaries, heaps, …
    data structure traversals, iterators, applicators, manipulators;
    searching (linear, binary, hashing); event handling; exceptions
- System level
  - Higher level
  - E.g. interface with OS, protocols, etc.

- Analysis of code without execution
- Code level:
  - Disassembly, control flow reconstruction (non-trivial)
  - Decompilation (reconstruction of higher level constructs, undecidable)
  - Detection (vulnerabilities, etc)
- System level
  - Header information (meta data)
  - Imported libraries, functions
  - Resources (fixed locations for embedded data, GUI, etc)

- Analyzing program executions (at runtime)
- Observe target program's behavior
  - Execute program (requires input/test data)
  - Potentially dangerous (malware?, side effects) -> VM
  - Emulator, etc. for legacy software
  - See program testing lecture
- Code level
  - Debugging (manual, user mode / kernel level) vs. (semi-)automatic (fuzzing, symbolic execution)
- System level (automatic, efficient)
  - Monitoring: system and library calls
  - Process creation, file system, network

- **Evolution of x86**
- 16 bit architecture 8086 and 8088 (only 8 bit bus)
  - Real mode (only), memory segmentation
  - 80286: protected mode
- 32 bit architecture (ia32)
  - 80386: 32 bit registers and protected mode address space
  - Pentium family introduced MMX + MMX registers, SIMD extensions
- 64 bit architecture (ia64)
  - Pentium 4 / AMD Athlon 64: 32 bit registers and address space
  - Removes certain instructions of 16/32 bit cores

- 32 bit variant or extension in parentheses

- Instruction pointer: (e)ip
- Stack pointer: (e)sp
- Base pointer: (e)bp
- General purpose: (e)ax, (e)bx, (e)cx, (e)dx, (e)si, (e)di
  further split of ax into al and ah, same for b,c,d
- Segment registers: cs, ds, ss, es, (fs, gs)
- Floating point: 8 register stack (st(0) to st(7))
- eflags: 16 bit for CF, PF, AF, ZF, SF, 0F, TF, IF, DF
- (SIMD registers: mmx0 to mmx7)

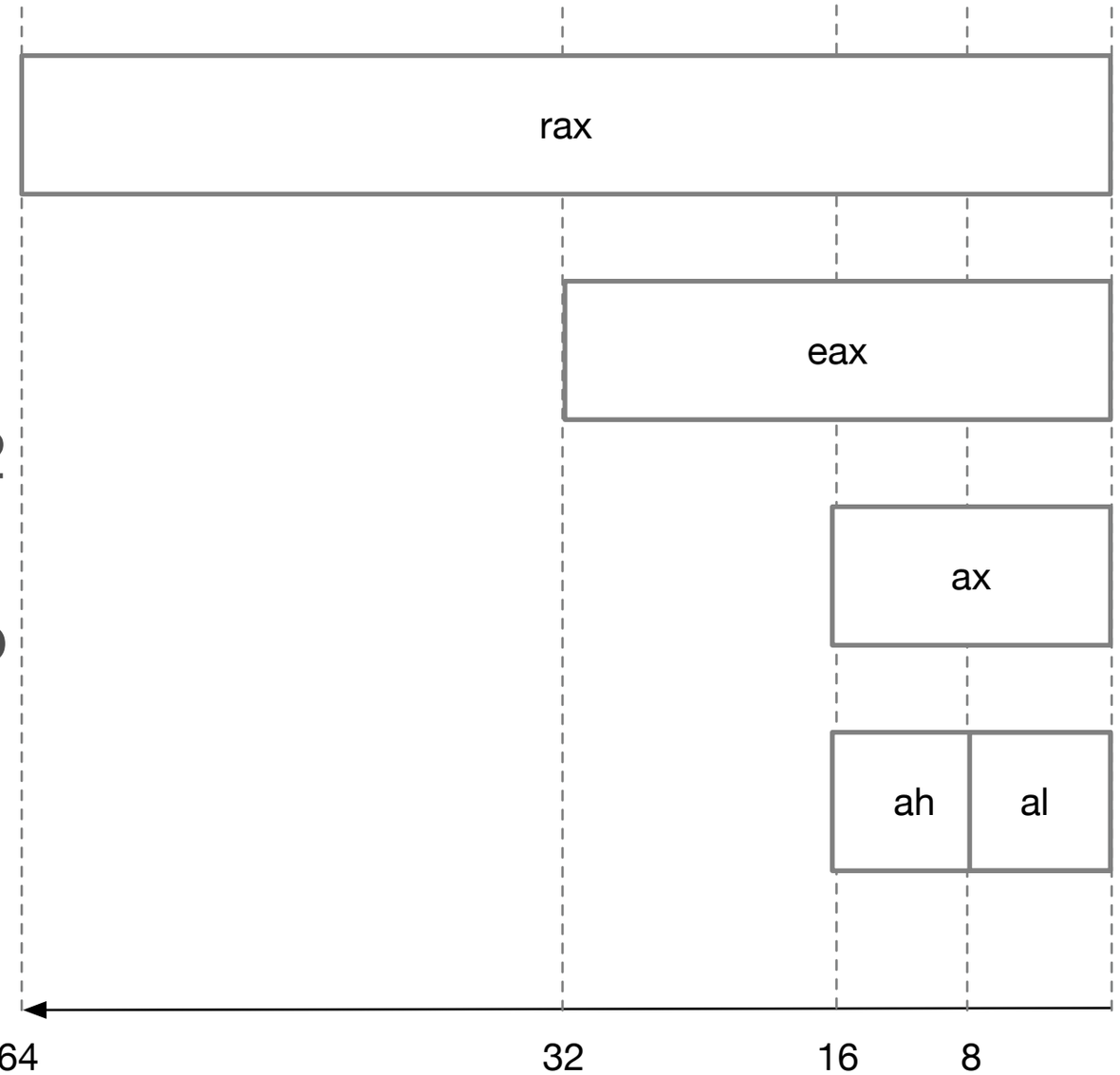
- Instruction pointer: rip
- Stack pointer: rsp
- Base pointer: rbp
- General purpose: rax, rbx, rcx, rdx, rsi, rdi, r8, … , r15
  further split of ax into al and ah, same for b,c,d
- Segment registers: cs, ds, ss, es, fs, gs
- Floating point: 8 register stack (st(0) to st(7))
- eflags: 16 bit for CF, PF, AF, ZF, SF, 0F, TF, IF, DF
- SIMD registers: xmm0 to xmm15

- Differs from previous Intel processors (16/32 bit)

- RAX, RCX, RDX, R8, R9, R10, R11, and XMM0-XMM5 are considered volatile (potentially changed by callee on return)
- RBX, RBP, RDI, RSI, RSP, R12, R13, R14, R15, and XMM6-XMM15 nonvolatile: must be saved and restored by a function using them
- Integer arguments passed in registers RCX, RDX, R8, and R9, space must be reserved on stack for callee to spill them, further parameters are passed on the stack in right-to-left order
- Floating point arguments are passed in XMM0L, XMM1L, XMM2L, and XMM3L
- Return value stored in RAX or XMM0

Accessing “subregisters”

- rax to rdx allow accessing fragments of registers, even high byte of 16 bit word
- rsi, esi, si, sil
- r8, r8d, r8w, r8b
- Backwards compatibility:
  - Writing eax zeroes significant 32 bit of rax
  - Writing ax, al, ah does NOT zero significant parts (or al) of “superregister” (eax or ax)



- RISC (e.g. MIPS)
  - Many registers, typically 32
  - Few simple addressing modes
  - Load-/store-architecture
  - three-address code: Rz ← Rx ⊕ Ry
  - constant-length instruction encoding, typically 4 bytes

- CISC
  - Fewer registers, 8–16
  - Complex address modes
  - Memory operands
  - two-address code: Rx ← Rx ⊕ Ry
  - variable-length instruction encoding (x86: from 1 to 15 bytes)

- Consist of *opcode* and zero or more *operands*
- *Opcode* defines operation (move data, call a function, …)
- Operands are parameters, three classes supported
  - Register (read or write)
  - Immediate (constant)
  - Memory address: constant (direct) or computed (indirect)

- mov dword ptr [0xdeafbeef], ebx
  store ebx into 4 bytes at given address in virtual memory
- mov ah, byte ptr [0x100]
  load 1 byte at address into ah

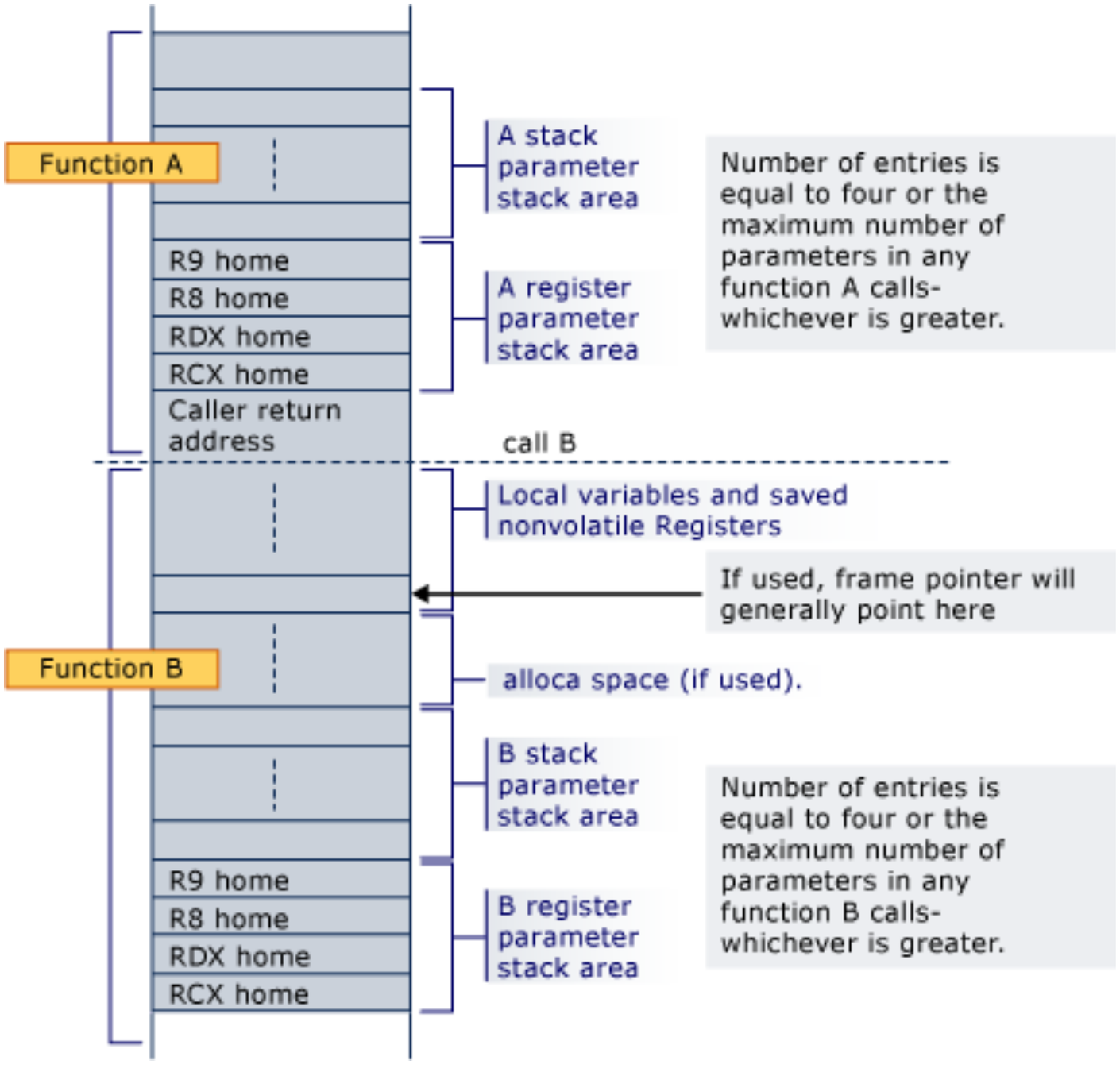
- CISC ISA
- 8 Registers (64-bit mode 16 registers)
- Powerful addressing modes:
  base register + (1,2,4) * index register + constant
- For many instructions, one operand can be a memory cell
  (instead of register)
- Inhomogeneous register usage:
  some registers only work with some instructions
- Hundreds of instructions in vector extensions
- Base, index, scale, displacement format: quickly access array elements

```
int *A;                          # ebx = A, ecx = i
...                              mov eax, 100
A[i+2] += 100                    add [ebx + ecx*4 + 8], eax
```

- Arithmetic
  - add/sub op1, op2
  - mul op: multiply operand by eax, store into edx:eax
  - div op: divide edx:eax by op, store into eax, remainder into edx
  - imul, idiv: signed couterparts
- Bitvector
  - and/or/xor op1, op2
  - not op
  - xor op, op: equivalent to but shorter than mov op, 0
- Unconditional jump:
  - jmp, call addr: (direct or indirect) jump or call
  - ret const: return and increase stack pointer by const

- Address-of operation
  - lea reg, mem: set reg to address of mem, e.g., lea eax, [edx*4 + ebp]
- Stack
  - push op
  - pop op
- Comparison:
  - cmp op1, op2: sets flags for op1 - op2
  - test op1, op2: sets flags for op1 & op2, e.g., test eax, eax
- Cond. jump
  - je/jz addr: jump if equal (i.e. zero)
  - jne/jnz, jl, jg, jle, jge: not equal, less, greater (or equal)

Call Stack



- A calling B: Prolog, Epilog

- mov [RSP + 8], RCX
  push R15
  push R14
  push R13
  sub RSP, 0x40 ; fixed-allocation-size
  lea RBP, 64[RSP]

- lea RSP, -64[RBP]
  ; epilogue proper starts here
  add RSP, 0x40 ; fixed-allocation-size
  pop R13
  pop R14
  pop R15
  ret

- William C. Chu et al.: Reverse Engineering, in Handbook Of Software Engineering And Knowledge Engineering, Vol 2: Emerging Technologies, 2002, World Scientific
- Liam O’Brien: Analysis of Software Artifacts – Reverse Engineering, Lecture at CMU, 2005
- J. Kinder: Reverse Engineering, Lecture at UBW, 2020
- Microsoft Documentation for Visual Studio 2022